



Introducción a ASP.NET Core

¿Qué es ASP.NET Core?

ASP.NET Core es un framework moderno, multiplataforma y de alto rendimiento desarrollado por Microsoft para crear aplicaciones web, APIs y servicios empresariales.

Características Principales

- Multiplataforma (Windows, Linux, macOS)
- Open Source
- Alto rendimiento
- Arquitectura basada en Middleware
- Inyección de Dependencias integrada
- Preparado para la nube y contenedores

Estructura de un Proyecto ASP.NET Core

- Controllers: Manejan las solicitudes HTTP
- Models: Representan los datos
- Views: Interfaz de usuario (MVC)
- wwwroot: Archivos estáticos (CSS, JS, imágenes)
- Program.cs: Configuración principal
- appsettings.json: Configuración del proyecto

Flujo de Funcionamiento

Cuando un usuario hace una petición desde el navegador, el servidor Kestrel recibe la solicitud, pasa por el pipeline de Middlewares, el sistema de Routing encuentra el Controller adecuado, se ejecuta la acción y finalmente se devuelve una respuesta.

Middleware (Pipeline)

Que es el Pipeline:

- El pipeline es el flujo de procesamiento de una solicitud HTTP dentro de la aplicación.
- El pipeline es la secuencia ordenada de middlewares que procesan una solicitud HTTP en ASP.NET Core.

ASP.NET Core trabaja con un pipeline de middlewares.

- Un middleware es como un filtro o estación de control por donde pasa cada petición antes de llegar al controlador.

- Un middleware es un componente que forma parte del pipeline HTTP y que puede interceptar, procesar y modificar solicitudes y respuestas en ASP.NET Core.

Cada middleware puede:

Middleware	Función
UseHttpsRedirection()	Redirige HTTP a HTTPS
UseStaticFiles()	Sirve archivos estáticos
UseRouting()	Activa el sistema de rutas
UseAuthorization()	Valida permisos

Ejemplos de Middlewares comunes:

- Permitir continuar
- Bloquear la petición
- Modificar la respuesta
- Terminar el proceso

Orden de ejecución de los Middlewares

ASP.NET Core usa un pipeline secuencial.

Funciona así:

1. La petición entra por el servidor web Kestrel
2. Pasa por cada middleware en orden
3. Si ninguno la detiene, llega al controlador
4. La respuesta regresa por el pipeline

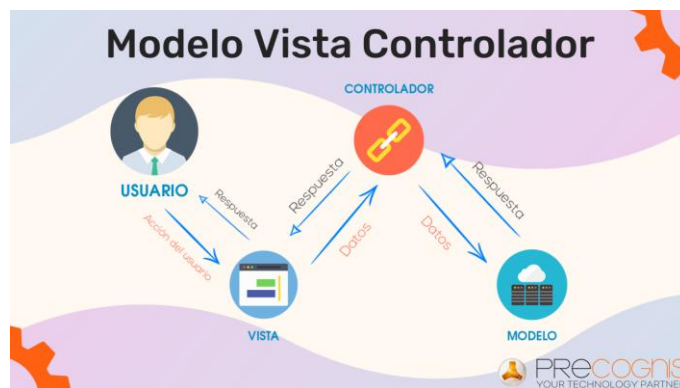
Modelo MVC

ASP.NET Core puede trabajar con el patrón:

Model–View–Controller

Donde:

- **Model** → Lógica y datos
- **View** → Interfaz
- **Controller** → Maneja las solicitudes



¿Qué son los Servicios en ASP.NET Core?

En ASP.NET Core, un **servicio** es una clase que contiene lógica reutilizable y que se registra en el contenedor de dependencias para que el sistema la administre automáticamente.

Ejemplos de servicios:

- Acceso a base de datos
- Lógica de negocio
- Envío de correos
- Autenticación
- Logging

¿Qué es la Inyección de Dependencias (DI)?

La Inyección de Dependencias es un patrón donde:

- Una clase no crea sus propias dependencias, sino que las recibe desde afuera.

Practica 1 de Introducción.

- Creación de Modelos.

```
public class Compra
{
    [Key]
    public int Id { get; set; }

    [Required]
    public DateTime FechaCompra { get; set; } = DateTime.Now;

    [Required]
    public int UsuarioId { get; set; }

    [ForeignKey("UsuarioId")]
    public Usuario Usuario { get; set; }

    [Required]
    public int VideoJuegoId { get; set; }

    [ForeignKey("VideoJuegoId")]
    public VideoJuego VideoJuego { get; set; }
}
```

```
public class Usuario
{
    [Key]
    public int Id { get; set; }

    [Required]
    [StringLength(100)]
    public string Nombre { get; set; }

    [Required]
    [EmailAddress]
    [StringLength(150)]
    public string Email { get; set; }

    [Required]
    [StringLength(255)]
    public string Password { get; set; }

    [Required]
    public DateTime FechaRegistro { get; set; } = DateTime.Now;

    public ICollection<Compra> Compras { get; set; }
}
```

```
public class VideoJuego
{
    [Key]
    public int Id { get; set; }

    [Required]
    [StringLength(150)]
    public string Titulo { get; set; }

    [Required]
    [Column(TypeName = "decimal(10,2)")]
    public decimal Precio { get; set; }

    [Required]
    [StringLength(100)]
    public string Categoria { get; set; }

    [StringLength(500)]
    public string Descripcion { get; set; }

    public ICollection<Compra> Compras { get; set; }
}
```

- Instalación de paquetes desde NuGet:
 1. Microsoft.EntityFrameworkCore
 2. Microsoft.EntityFrameworkCore.SqlServer
 3. Microsoft.EntityFrameworkCore.Tools

- Creación del servicio de conexión a base de datos.

```
public class TiendaContext : DbContext
{
    public TiendaContext(DbContextOptions<TiendaContext> options)
        : base(options)
    {
    }

    public DbSet<Usuario> Usuarios { get; set; }
    public DbSet<VideoJuego> VideoJuegos { get; set; }
    public DbSet<Compra> Compras { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Usuario>()
            .HasIndex(u => u.Email)
            .IsUnique();

        modelBuilder.Entity<Compra>()
            .HasOne(c => c.Usuario)
            .WithMany(u => u.Compras)
            .HasForeignKey(c => c.UsuarioId)
            .OnDelete(DeleteBehavior.Cascade);

        modelBuilder.Entity<Compra>()
            .HasOne(c => c.VideoJuego)
            .WithMany(v => v.Compras)
            .HasForeignKey(c => c.VideoJuegoId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}
```

- Configuración de la base de datos.

```
{  
  "ConnectionStrings": {  
    "DefaultConnection":  
    "Server=.;Database=TiendaVideoJuegosDB;Trusted_Connection=True;TrustServer  
Certificate=True;"  
  },  
}
```

- Registro del servicio.

```
builder.Services.AddDbContext<TiendaContext>(options =>  
    options.UseSqlServer(  
        builder.Configuration.GetConnectionString("DefaultConnection")  
    )  
);
```

- Ejecución de migraciones y empuje de base de datos.

1. Add-Migration Inicial
2. Update-Database